

Arquitectura de Datos

Tema 3: Redis

Bases de datos clave-valor en memoria

Curso académico 2026/2027





Objetivo

Domina las **5 estructuras principales** de Redis: Strings, Hashes, Lists, Sets y Sorted Sets. Cada una está optimizada para un caso de uso concreto y ofrece operaciones con complejidad constante, mejor que las tablas hash tradicionales.

Al finalizar este laboratorio serás capaz de:

- Almacenar y manipular valores simples con **Strings** (SET, GET, APPEND, STRLEN, INCR, MSET, MGET).
- Representar objetos con múltiples campos usando **Hashes** (HSET, HGET, HGETALL, HMGET, HINCRBY, HDEL, HLEN).
- Gestionar colas e historiales con **Lists** (LPUSH, RPUSH, LPOP, RPOP, LRANGE, LLEN, LINDEX).
- Trabajar con conjuntos sin duplicados usando **Sets** (SADD, SMEMBERS, SCARD, SISMEMBER, SINTER, SUNION, SDIFF).
- Implementar rankings y leaderboards con **Sorted Sets** (ZADD, ZRANGE, ZREVRANGE, ZRANK, ZSCORE, ZINCRBY, ZCARD).

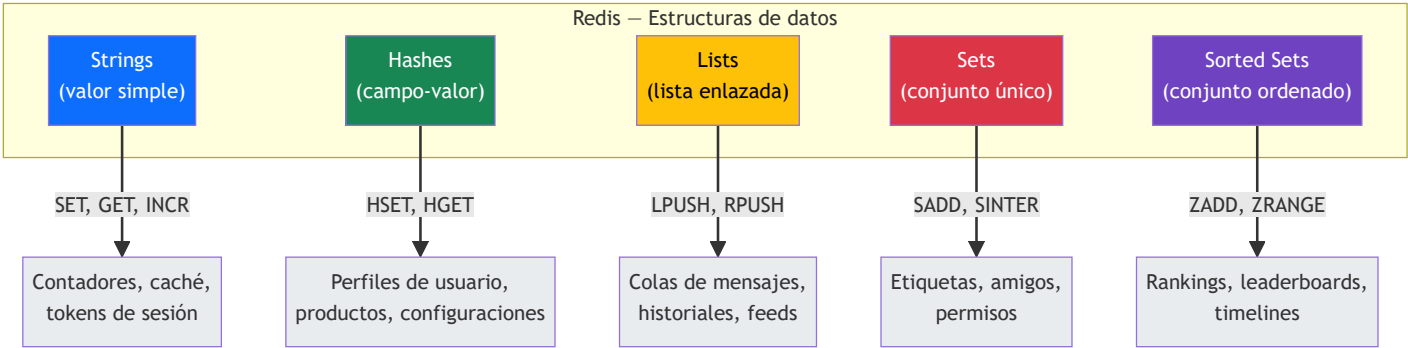
Requisitos previos

Componente	Versión mínima	Comprobación
Docker Desktop	4.x	<code>docker --version</code>
JetBrains DataGrip	2024.x	Abrir la aplicación
Redis (imagen Docker)	7.x	<code>docker pull redis:7-alpine</code>
Práctica PR1 completada	—	Contenedor Redis en ejecución

Nota: Antes de comenzar, verifica que el contenedor Redis de la Práctica PR1 está ejecutándose con `docker ps`. Si no está activo, levántalo con `docker-compose up -d` desde el directorio del proyecto.

Arquitectura de las estructuras de datos

Redis almacena todos los datos en memoria RAM y ofrece 5 estructuras de datos fundamentales. Cada una está diseñada para un caso de uso específico y todas comparten la propiedad de ser extremadamente rápidas (complejidad $O(1)$ en la mayoría de operaciones).



Explicación: Cada estructura de Redis resuelve un problema distinto. Los Strings son el tipo más versátil. Los Hashes optimizan el almacenamiento de objetos con múltiples campos. Las Lists permiten inserción/eliminación eficiente por ambos extremos. Los Sets evitan duplicados y permiten operaciones de teoría de conjuntos. Los Sorted Sets añaden una puntuación numérica para mantener el orden automáticamente.

Paso 1: Strings — El tipo básico

Los Strings son el tipo más versátil de Redis. Permiten almacenar texto, números, bytes y datos serializados. Son la base sobre la que se construyen todas las demás operaciones.

1.1 SET y GET — Almacenamiento básico

Comando SET — Crear una clave

```
# Almacena el valor "Juan García" en la clave user:1000:name
# Si la clave ya existía, se sobrescribe
SET user:1000:name "Juan García"
```

Salida esperada:

Resultado

```
OK
```

Comando GET — Leer una clave

```
# Obtiene el valor almacenado en la clave user:1000:name
GET user:1000:name
```

Salida esperada:

Resultado

```
"Juan García"
```

1.2 APPEND y STRLEN — Manipulación de strings

Comando APPEND — Añadir texto al final

```
# Añade " López" al final del string existente
# Devuelve la longitud resultante del string
APPEND user:1000:name " López"
```

Salida esperada:

Resultado

```
(integer) 15
```

Comando STRLEN – Obtener la longitud

```
# Devuelve la longitud del string almacenado
STRLEN user:1000:name
```

Salida esperada:

Resultado

```
(integer) 15
```

1.3 INCR y INCRBY — Contadores atómicos

Comando INCR – Incrementar en 1

```
# Inicializar un contador de visitas en 0
SET product:456:views 0

# Incrementar el contador en 1 (operación atómica)
INCR product:456:views
```

Salida esperada:

Resultado

```
(integer) 1
```

Comando INCRBY – Incrementar en una cantidad

```
# Incrementar el contador en 10 unidades
INCRBY product:456:views 10
```

Salida esperada:

Resultado

```
(integer) 11
```

1.4 MSET y MGET — Operaciones múltiples

Comando MSET — Crear varias claves de una vez

```
# Almacena múltiples pares clave-valor en una sola operación
# Más eficiente que múltiples SET individuales (1 roundtrip vs N)
MSET user:1000:name "Juan" user:1000:email "juan@email.com" user:1000:city "Madrid"
```

Salida esperada:

Resultado

```
OK
```

Comando MGET — Leer varias claves

```
# Obtiene los valores de múltiples claves en una sola operación
# Devuelve una lista con los valores (nil si una clave no existe)
MGET user:1000:name user:1000:email user:1000:city
```

Salida esperada:

Resultado

```
"Juan" → "juan@email.com" → "Madrid"
```

Explicación: Los contadores INCR e INCRBY son **atómicos**. Redis ejecuta todos los comandos en un único hilo, por lo que múltiples clientes concurrentes nunca obtienen resultados inconsistentes. Es la misma propiedad que utilizan los bancos para contadores de transacciones.

Paso 2: Hashes — Mapas campo-valor

Los Hashes almacenan pares **campo-valor** dentro de una clave. Son ideales para representar objetos con múltiples propiedades como perfiles de usuario o productos, ya que permiten acceder a campos individuales sin descargar el objeto completo.

2.1 HSET y HGET — Campos individuales

Comando HSET — Establecer campos en un hash

```
# Establecer campos individuales en un hash
HSET user:1000 name "Juan García"
HSET user:1000 email "juan@email.com"
HSET user:1000 age 28
HSET user:1000 city "Madrid"
```

Salida esperada:

Resultado

```
(integer) 1
```

Comando HGET — Obtener un campo específico

```
# Obtener solo el campo "name" del hash
HGET user:1000 name
```

Salida esperada:

Resultado

```
"Juan García"
```

2.2 HGETALL y HMGET — Obtener múltiples campos

Comando HGETALL — Obtener todos los campos

```
# Obtener todos los campos y valores del hash
HGETALL user:1000
```

Salida esperada:

Resultado

```
"name"    → "Juan García"
"email"    → "juan@email.com"
"age"      → "28"
"city"     → "Madrid"
```

Comando HMGET – Obtener varios campos a la vez

```
# Obtener valores de varios campos específicos
HMGET user:1000 name email
```

Salida esperada:

Resultado

```
"Juan García" → "juan@email.com"
```

2.3 HINCRBY, HDEL y HLEN

Comando HINCRBY – Incrementar un campo numérico

```
# Incrementar el valor numérico del campo "age" en 1
HINCRBY user:1000 age 1
```

Salida esperada:

Resultado

```
(integer) 29
```

Comando HDEL – Eliminar un campo

```
# Eliminar el campo "city" del hash
HDEL user:1000 city
```

Salida esperada:

Resultado

```
(integer) 1
```

Comando HLEN – Contar los campos

```
# Contar el número de campos del hash  
HLEN user:1000
```

Salida esperada:

Resultado

```
(integer) 3
```

Explicación: Los Hashes son más eficientes que múltiples claves String cuando un objeto tiene varios campos. Permiten actualizar un solo campo sin tocar los demás, y HGETALL recupera el objeto completo en una sola operación. El comando HINCRBY es atómico, igual que INCR para Strings.

Paso 3: Lists — Listas enlazadas

Las Listas son **listas enlazadas bidireccionales** que permiten inserción y eliminación eficiente por ambos extremos con complejidad $O(1)$. Son ideales para colas FIFO, pilas LIFO e historiales.

3.1 LPUSH y RPUSH — Insertar elementos

Comando LPUSH — Insertar por la izquierda

```
# Insertar por la izquierda (inicio de la lista)
LPUSH colas:tareas "tarea1"
```

Salida esperada:

Resultado

```
(integer) 1
```

Comando RPUSH — Insertar por la derecha

```
# Insertar por la derecha (final de la lista)
RPUSH colas:tareas "tarea2"
```

Salida esperada:

Resultado

```
(integer) 2
```

Comando LPUSH — Insertar otro elemento

```
# Insertar otro elemento por la izquierda
LPUSH colas:tareas "tarea0"
```

Salida esperada:

Resultado

```
(integer) 3
```

3.2 LPOP y RPOP — Extraer elementos

Comando LPOP — Extraer por la izquierda (FIFO)

```
# Extraer por la izquierda: primero en entrar, primero en salir
LPOP colas:tareas
```

Salida esperada:

Resultado

```
"tarea0"
```

Comando RPOP — Extraer por la derecha (LIFO)

```
# Extraer por la derecha: último en entrar, primero en salir
RPOP colas:tareas
```

Salida esperada:

Resultado

```
"tarea2"
```

3.3 LRANGE, LLEN y LINDEX — Consultar la lista

Comando LRANGE — Obtener un rango de elementos

```
# Obtener todos los elementos (0 = primero, -1 = último)
LRANGE colas:tareas 0 -1
```

Salida esperada:

Resultado

```
"tarea1"
```

Comando LLEN – Obtener la longitud

```
# Contar el número de elementos de la lista
LLEN colas:tareas
```

Salida esperada:

Resultado

```
(integer) 1
```

Comando LINDEX – Obtener un elemento por índice

```
# Obtener el elemento en la posición 0 (el primero)
LINDEX colas:tareas 0
```

Salida esperada:

Resultado

```
"tarea1"
```

Explicación: Las Listas de Redis son listas enlazadas, no arrays. Esto significa que LPUSH y RPUSH tienen complejidad $O(1)$, mientras que LINDEX con un índice aleatorio puede ser $O(N)$. Para colas de mensajes, usa siempre LPUSH + RPOP (patrón FIFO).

Paso 4: Sets — Conjuntos sin duplicados

Los Sets son **conjuntos desordenados** de strings únicos. No permiten duplicados y soportan operaciones de teoría de conjuntos: intersección, unión y diferencia. Son ideales para etiquetas, amigos o permisos.

4.1 SADD — Añadir miembros

Comando SADD — Añadir miembros al conjunto

```
# Añadir miembros al conjunto (los duplicados se ignoran)
SADD likes:post:1 "user:1000"
# → 1 (añadido correctamente)

SADD likes:post:1 "user:1001"
# → 1

SADD likes:post:1 "user:1002"
# → 1

SADD likes:post:1 "user:1000"
# → 0 (duplicado, no se añade)
```

Salida esperada (cada comando):

Resultado

```
(integer) 1
(integer) 1
(integer) 1
(integer) 0
```

4.2 SMEMBERS, SCARD y SISMEMBER — Consultas

Comando SMEMBERS — Obtener todos los miembros

```
# Obtener todos los miembros del conjunto
SMEMBERS likes:post:1
```

Salida esperada:

Resultado

```
"user:1000" → "user:1001" → "user:1002"
```

Comando SCARD – Contar miembros

```
# Contar el número de miembros del conjunto  
SCARD likes:post:1
```

Salida esperada:

Resultado

```
(integer) 3
```

Comando SISMEMBER – Verificar pertenencia

```
# Verificar si un miembro pertenece al conjunto (1 = sí, 0 = no)  
SISMEMBER likes:post:1 "user:1000"
```

Salida esperada:

Resultado

```
(integer) 1
```

Comando SISMEMBER – Miembro inexistente

```
# Verificar un miembro que no está en el conjunto  
SISMEMBER likes:post:1 "user:9999"
```

Salida esperada:

Resultado

```
(integer) 0
```

4.3 SINTER, SUNION y SDIFF — Operaciones de conjuntos

Preparar segundo conjunto

```
# Crear un segundo conjunto de Likes
SADD likes:post:2 "user:1001"
SADD likes:post:2 "user:1002"
SADD likes:post:2 "user:1003"
```

Comando SINTER — Intersección

```
# Usuarios que dieron Like en AMBOS posts (intersección)
SINTER likes:post:1 likes:post:2
```

Salida esperada:

Resultado

```
"user:1001" → "user:1002"
```

Comando SUNION — Unión

```
# Usuarios que dieron like en CUALQUIER post (unión)
SUNION likes:post:1 likes:post:2
```

Salida esperada:

Resultado

```
"user:1000" → "user:1001" → "user:1002" → "user:1003"
```

Comando SDIFF — Diferencia

```
# Usuarios que dieron like solo en post:1 (diferencia)
SDIFF likes:post:1 likes:post:2
```

Salida esperada:

Resultado

```
"user:1000"
```

Explicación: Los Sets son la representación de conjuntos matemáticos en Redis. SINTER calcula la intersección ($\cap$), SUNION la unión ($\cup$) y SDIFF la diferencia ($-$). Todos estos comandos son $O(N)$ donde N es el número de miembros del conjunto más pequeño.

Paso 5: Sorted Sets — Conjuntos ordenados

Los Sorted Sets son como los Sets pero con una **puntuación (score)** asociada a cada miembro. Los miembros se mantienen ordenados por puntuación de menor a mayor automáticamente. Son ideales para rankings, leaderboards y timelines.

5.1 ZADD — Añadir miembros con puntuación

Comando ZADD — Añadir miembros al sorted set

```
# Añadir miembros con puntuación al sorted set
ZADD leaderboard 1500 "player:1000"
# → 1 (añadido correctamente)

ZADD leaderboard 2300 "player:1001"
# → 1

ZADD leaderboard 1800 "player:1002"
# → 1

ZADD leaderboard 2300 "player:1003"
# → 1

# Añadir varios miembros de una vez
ZADD leaderboard 2100 "player:1004" 1900 "player:1005"
# → 2
```

Salida esperada (cada comando):

Resultado

```
(integer) 1
(integer) 1
(integer) 1
(integer) 1
(integer) 2
```

5.2 ZRANGE y ZREVRANGE — Consultar el ranking

Comando ZRANGE — De menor a mayor

```
# Obtener todos los miembros ordenados de menor a mayor puntuación
ZRANGE leaderboard 0 -1 WITHSCORES
```

Salida esperada:

Resultado

```
"player:1000" → "1500"  
"player:1002" → "1800"  
"player:1005" → "1900"  
"player:1004" → "2100"  
"player:1001" → "2300"  
"player:1003" → "2300"
```

Comando ZREVRANGE — Top 3 de mayor a menor

```
# Obtener el top 3 de mayor a menor (los mejores jugadores)  
ZREVRANGE leaderboard 0 2 WITHSCORES
```

Salida esperada:

Resultado

```
"player:1001" → "2300"  
"player:1003" → "2300"  
"player:1004" → "2100"
```

5.3 ZRANK y ZSCORE — Puntuación y posición

Comando ZRANK — Posición de menor a mayor

```
# Obtener la posición (rank) de un miembro (0 = el de menor puntuación)  
ZRANK leaderboard "player:1000"
```

Salida esperada:

Resultado

```
(integer) 0
```

Comando ZSCORE — Obtener la puntuación

```
# Obtener la puntuación de un miembro específico
ZSCORE leaderboard "player:1000"
```

Salida esperada:

Resultado

```
"1500"
```

5.4 ZINCRBY y ZCARD — Incrementar y contar

Comando ZINCRBY — Incrementar la puntuación

```
# Incrementar la puntuación de un miembro en 100 puntos
ZINCRBY leaderboard 100 "player:1000"
```

Salida esperada:

Resultado

```
"1600"
```

Comando ZCARD — Contar miembros

```
# Contar el número total de miembros del sorted set
ZCARD leaderboard
```

Salida esperada:

Resultado

```
(integer) 6
```

Explicación: Los Sorted Sets mantienen los miembros ordenados automáticamente por puntuación. ZRANGE recupera de menor a mayor y ZREVRANGE de mayor a menor. ZINCRBY es atómico, ideal para actualizar puntuaciones en tiempo real sin bloquear otros clientes.

Preguntas de reflexión

Antes de continuar con el siguiente laboratorio, reflexiona sobre las siguientes preguntas:

1. ¿Qué estructura usarías para almacenar un perfil de usuario con múltiples campos (nombre, email, ciudad, fecha de registro)?

Compara las ventajas de usar un Hash frente a múltiples claves String. ¿Qué impacto tiene en el número de operaciones y en la consistencia de los datos?

2. ¿Cuándo elegirías un Set frente a un Sorted Set?

Piensa en un caso de uso donde solo necesitas saber si un elemento pertenece a un conjunto (Set) y en otro donde necesitas mantener un orden. ¿Qué sobrecoste implica el Sorted Set?

3. ¿Por qué las Lists son ideales para implementar colas de mensajes?

Analiza la complejidad de las operaciones LPUSH y RPOP. ¿Qué patrón de acceso se crea y por qué es eficiente para el procesamiento de mensajes en segundo plano?

4. ¿Qué estructura usarías para implementar un ranking de usuarios por puntuación en un videojuego?

Considera las operaciones necesarias: añadir jugadores, actualizar puntuaciones, obtener el top 10 y consultar la posición de un jugador específico. ¿Qué estructura satisface todas estas necesidades con la mejor complejidad?

5. ¿Cómo guardarías la configuración de un usuario con TTL por campo?

Redis no permite expirar campos individuales de un Hash. ¿Cómo podrías combinar Strings con expiración y Hashes para lograr un TTL por campo? ¿Qué trade-offs implica cada enfoque?

Comandos de referencia rápida

Comando	Estructura	Descripción	Ejemplo
SET	String	Crear/actualizar una clave	SET clave valor
GET	String	Leer el valor de una clave	GET clave
APPEND	String	Añadir texto al final	APPEND clave texto
INCR	String	Incrementar en 1	INCR contador
MSET	String	Crear múltiples claves	MSET k1 v1 k2 v2
MGET	String	Leer múltiples claves	MGET k1 k2 k3
HSET	Hash	Establecer un campo	HSET hash campo valor
HGET	Hash	Obtener un campo	HGET hash campo
HGETALL	Hash	Obtener todos los campos	HGETALL hash
LPUSH	List	Insertar por la izquierda	LPUSH lista valor
RPUSH	List	Insertar por la derecha	RPUSH lista valor
LRANGE	List	Obtener un rango	LRANGE lista 0 -1
SADD	Set	Añadir miembros	SADD set miembro
SMEMBERS	Set	Obtener todos los miembros	SMEMBERS set
SINTER	Set	Intersección de conjuntos	SINTER set1 set2
ZADD	Sorted Set	Añadir con puntuación	ZADD zset score miembro
ZRANGE	Sorted Set	Rango de menor a mayor	ZRANGE zset 0 -1
ZSCORE	Sorted Set	Obtener la puntuación	ZSCORE zset miembro



Arquitectura de Datos

4.º Curso, Grado en Ingeniería Informática

Curso académico 2026/2027

Human Language and Accessibility Technologies Group (HULAT).

Departamento de Informática

Universidad Carlos III de Madrid